# Model Evaluation Plan: Evaluating ASR Robustness Under Overlapping English Speech

Jamin

April 22, 2026

# Contents


**1 Introduction** **3**

**2 Literature Review** **5**
2.1 Foundational Knowledge . . . 5
2.1.1 Multi-Speaker Corpora for ASR Research . . . 5
2.1.2 Overlap Modelling and Evaluation Metrics for Multi-Speaker ASR . . . 6
2.1.3 ASR Architectures and Benchmark Systems for Robust Speech Transcription 6
2.2 Establishing a Gap in the Literature . . . 7
2.3 Recent Advances . . . 7
2.3.1 Realism and Generalisation in Overlap Modelling . . . 7
2.3.2 Integrated Pipelines and End-to-End ASR for Overlapping Speech . . . 7
2.3.3 Unified Metrics for Multi-Speaker ASR Evaluation . . . 8
2.3.4 Summary . . . 9

**3 Architectural Design** **10**
3.1 System Overview . . . 10
3.2 Synthetic Mixture Generation . . . 10
3.2.1 Speaker and Clip Selection . . . 11
3.2.2 Offset Scheduling . . . 11
3.2.3 Noise Addition . . . 12
3.2.4 Manifest Structure . . . 12

**4 System and Data Flow** **14**
4.1 Mixture Generation Algorithm . . . 14
4.2 Representative ASR Models . . . 17
4.3 Evaluation Metrics . . . 17

**5 System Architecture and Implementation** **19**
5.0.1 Libraries and Tools . . . 19
5.1 Synthetic Mixture Generation . . . 19
5.2 ASR Decoding . . . 20
5.3 Error Metrics: WER, cpWER, DI-cpWER . . . 20

**6 Datasets, ASR Models and Preprocessing Strategies** **21**
6.1 Datasets . . . 21
6.2 Baseline Models under Evaluation . . . 21
6.3 Preprocessing Strategies . . . 22

**7 Experimental Design** **23**
7.1 Objective and Scope . . . 23
7.2 Research Questions . . . 23
7.3 Experimental Design Overview . . . 23

7.3.1 Factors and Levels . . . 24
7.3.2 Performance Metrics . . . 25
7.3.3 Experimental Units . . . 25
7.4 Datasets and Data Splits . . . 26
7.4.1 Track A: Synthetic Mixture Dataset (Controlled Overlap) . . . 26
7.4.2 Track B: Real Overlapped Speech Dataset . . . 26

**8 Preprocessing Pipelines** **27**
8.1 Preprocessing Structure . . . 27
8.1.1 Pipeline Definition . . . 27
8.1.2 Experimental Plan . . . 27
8.2 ASR Inference Controls . . . 27
8.3 Evaluation Protocol . . . 28
8.3.1 Reference Transcripts . . . 28
8.3.2 Primary and Secondary Metrics . . . 28
8.3.3 Overlap-aware Evaluation . . . 28
8.3.4 cpWER for Multi-speaker References . . . 28

**9 Evaluation Metrics** **29**
9.1 Overview and Principles . . . 29
9.2 Response Variables . . . 29
9.3 Bootstrap Confidence Intervals . . . 29
9.4 Paired Comparisons (Preprocessing Effects) . . . 29
9.5 Main Effects and Interactions (Model-Based Analysis) . . . 30
9.6 Reporting of Effect Sizes . . . 30
9.7 Transferability: Synthetic vs Real (RQ3) . . . 30

**10 Statistical Analysis Plan** **31**

# Chapter 1

# Introduction

Automatic speech recognition (ASR) has improved substantially in recent years due to advances in deep learning, large-scale training data, and computational resources. However, performance remains much less reliable in realistic conversational settings, particularly when multiple speakers overlap, and background noise is present. To develop and evaluate models under these conditions, robust, realistic datasets are needed. The AMI meeting corpus [1] and the CHIME-6 dataset [2] provide important examples of real-world multi-speaker datasets with realistic, detailed annotations.

Two major factors that degrade ASR performance are overlapping speech and environmental noise. Both are common in natural human conversation [3, 4]. Overlapping speech is the time interval during which two or more speakers speak simultaneously. At the same time, background noise is any unintended sound that does not contribute to the conversation. Overlapping speech violates the single-active-speaker assumption underlying many conventional ASR systems. Natural conversations often involve single-speaker segments and partially overlapping regions [3], where standard metrics such as word error rate (WER) might fail to reflect the accuracy [5, 6]. AS a result, overlap-aware and speaker-aware measures such as concatenated minimum-permutation WER (cpWER), time-constrained cpWER (tcpWER), and diarisation invariant cpWER (DI-cpWER) are proposed [7].

Due to the difficulties and cost of collecting and annotating real-world multi-speaker datasets, synthetic mixtures have emerged as an important alternative. Existing datasets have improved realism by adding ambient noise, reverberation and increasing the diversity of recording conditions and scenarios. However, most were fixed rather than flexible for downstream training to systematically produce more data and varying overlap, noise, duration and speaker count.

Despite this progress, a practical gap remains. Real meeting conversations preserve natural patterns, but this same naturalness poses lexical and acoustic challenges, limiting control over the degradation factors during training and evaluation. On the other hand, existing synthetic datasets are easier to manipulate, but they are typically built with fixed or partially controlled parameters. As a result, it remains difficult to isolate how factors such as overlap ratio, signal-to-noise ratio, mixture duration and speaker count individually and jointly affect ASR performance.

This dissertation addresses the gap by introducing a synthetic mixture generation framework that takes clean speaker recordings, transcription and noise recording as input and outputs mixtures with a user-specified overlap ratio, signal-to-noise ratio, audio length and speaker count, along with timestamped transcripts. Utilising this framework, this study investigates three main questions: how ASR performance changes with overlap and noise, how transferable is the trend observed in synthetic data to real-world, and how publicly available ASR models compare under different conditions.

This framework provides more transparency and information in model evaluation by separating different conditions by degradation factors. In the case where robustness changes with degradation conditions, single-point evaluation becomes underrepresentative. Hence, instead

of simply reporting under a single dataset, this framework provides an opportunity to analyse how different models behave in a range of different conditions, identifying the strengths and weaknesses of different models.

The structure of the dissertation is as follows: Chapter 2 reviews the related work on ASR robustness, multi-speaker datasets and evaluation metrics. Chapter 3 describes the datasets, ASR models and preprocessing strategies used in the experiments. Chapter 4 details the experimental design, including the generation of synthetic mixtures and the evaluation on real-world data. Chapter 5 outlines and reports the results of experiments. Chapter 6 discusses the implications of the findings, limitations of the study and directions for future work. Finally, Chapter 7 concludes the dissertation by summarising the key contributions and insights gained from the research.

# Chapter 2

# Literature Review

This chapter reviews the literature relevant to evaluating the robustness of automatic speech recognition (ASR) to overlapping speech. It is structured into three sections. The first section summarises established knowledge about multi-speaker datasets, synthetic mixture generation, overlap modelling, evaluation metrics and ASR architectures. The second section identifies the primary gaps motivating this dissertation. That includes the fact that existing resources do not jointly provide realistic overlap, factor-level control, and fair evaluation when comparing references and hypotheses with different structures. The third section explores recent work that has attempted to address these gaps through more realistic synthesis, more advanced pipelines and end-to-end ASR.

## 2.1 Foundational Knowledge

### 2.1.1 Multi-Speaker Corpora for ASR Research

Audio transcription research has depended on a relatively small set of corpora that preserve the acoustic and interactional realism of natural conversations. The *AMI Meeting Corpus* is one of the foundational resources for multi-speaker ASR because it provides approximately 100 hours of monitored meetings with rich annotations and a variety of naturally occurring and scenario-driven interactions [8]. The *CHiME-6* challenge further extended that problem into distant multi-microphone, unsegmented recordings in home environments, introducing a benchmark where diarisation and recognition are performed under noise, reverberation and spontaneous conversational turn-taking [9].

These datasets are foundational to ASR research because they retain the properties that are essential for downstream evaluation and application, like realistic microphone recording conditions, speaker interruptions, incomplete sentences and natural overlap. They together represent the trade-off between ecological validity and controlability. The real datasets preserve natural overlap patterns, acoustic effects and turn-taking, but they do not allow independent control of degradation factors. Inversely, synthetic datasets allow for precise control of factors but often lose the realism of natural conversation. This dissertation is designed to contribute to the middle ground, balancing realism and control.

Therefore, synthetic mixture generation has become a common technique in robust speech research. Earlier speech-separation datasets were often built by randomly mixing utterances under simplified conditions, but later datasets have progressively introduced realism by adding environmental noise, reverberation, corpus diversity and lexical continuity. *WHAM!* added real ambient noise and argued that clean, fully overlapped benchmarks were too narrow for realistic evaluation [10]. *WHAMR!* enhanced this by adding reverberation, modelling the spectral and temporal smearing that arises in indoor environments [11]. *LibriCSS* then provided a more generalised alternative to wsj0-2mix-based datasets like *WHAM!* with mixture derived from

*LibriSpeech* [3]. Finally, *LibriMix* extended this by mixing in VCTK speech, increasing the total length and thereby making it more suitable for training purposes [12]. The field has provided a rich set of resources for ASR research, but there is yet to be a framework that allows customised manipulation of overlap, noise, duration and speaker count in a reuseable way for generating synthetic mixtures.

### 2.1.2 Overlap Modelling and Evaluation Metrics for Multi-Speaker ASR

In this literature, the overlap ratio is a useful way to measure and control mixture difficulty. In LibriCSS, the overlap ratio is defined as the total duration of the overlapped region divided by the total speech duration [3]. This is methodologically important because natural conversation is not uniformly overlapped. It is often varied between different dialogue contexts [13]. A benchmark that reduces these contexts into a single average condition hides what a model is actually robust to and what it is not. The evolution from a simple fully overlapped mixture to realistically partially overlapped speech signified an important progress in realism, showing the increasing emphasis on treating overlap as an experimental variable rather than a marginal condition [14].

In ASR evaluation, the choice of metric can influence the weights of different aspects considered. In single-speaker ASR, word error rate (WER) is the standard metric, but in multi-speaker and overlapping settings, its calculation and interpretation are problematic. Galibert [15] shows that classical implementations of WER and diarisation error rate (DER) that were designed for single-speaker speech need to be generalised and adapted to multi-speaker conditions to remain meaningful. Several recent works have formalised the metric for multi-speaker transcription in different aspects. von Neumann et al. [16] shows that variants like cpWER can be understood within a wider framework of long-form multi-speaker WER, while von Neumann et al. [17] provides a toolkit for computing metrics like WER, cpWER, MIMO-WER and OCR-WER. von Neumann et al. [16] also discusses the separation between speaker-attributed metrics and speaker-agnostic metrics by comparing asclite, OCR-WER, cpWER and MIMO-WER. The conclusion is that lexical recognition, temporal information and speaker attribution are all important factors in multi-speaker ASR evaluation, and that no single metric captures all of these aspects equally well. As a result, the choice of metrics must therefore be aligned with the application and the specific aspect of robustness being evaluated while methods of comparing mismatched reference and hypothesis structures remain underexplored.

### 2.1.3 ASR Architectures and Benchmark Systems for Robust Speech Transcription

The model architecture operating on top of these datasets and metrics has evolved substantially. One family of models is the self-supervised encoders like wav2vec-2.0, which show that pretraining on very large unlabeled examples can learn representations that transfer effectively to downstream ASR by fine-tuning on smaller labelled datasets [18]. Another family is the large encoder-decoder models like Whisper, which uses weakly supervised pretraining and focuses on zero-shot transfer to improve robustness [19]. The third family is the conformer and transducer-based architectures like Parakeet that augmented the transformer with convolutional modules to capture both local patterns and global interactions and widely used in streaming ASR [20]. In modern multi-speaker ASR research, these models are often embedded inside complex pipelines that include front-end separation, speaker diarisation and transcription modules [21, 22]. In conclusion, the field already has mature corpora, mixture-generation, evaluation metrics and ASR models. However, the systematic framework for isolating the effects of overlap, noise and other factors under controlled environments remains less mature.

## 2.2 Establishing a Gap in the Literature

Despite the progress in multi-speaker ASR research, there are still important methodological gaps for this dissertation. Current real meeting corpora are ecologically valid, but they do not function as reusable testbeds in which overlap ratio, signal-to-noise ratio (SNR), clip duration, and speaker count can all be varied independently and systematically [1, 9]. On the other hand, synthetic datasets each solve only part of the problem. For example, WHAM! added noise, WHAMR! added reverberation, LibriMix provided more data and more realistic mixtures [10, 11, 12]. However, none of these resources was designed primarily as a configurable framework for causal analysis of ASR robustness. Hence, they provided many excellent benchmarks, but less support for answering narrow experimental questions about the standalone and interactive effects of degradation factors on ASR performance.

A second gap concerns the evaluation under structural mismatch between references and hypotheses. Recent metric work has clarified how to judge speaker-attributed hypotheses, speaker-agnostic hypotheses and multi-output systems [16, 17, 23]. However, in practice, many strong ASR systems still output a single sequential hypothesis even when the reference is multi-speaker, while overlap benchmark often encode reference speech as concurrent stream [18, 20]. This creates an awkward in-between case. While cpWER and tcpWER are designed for multi-stream references, they assume a speaker-structured hypothesis. ORC-WER is designed to ignore speaker labels and isolate lexical correctness. However, what is not provided is an estimation algorithm tailored to the specific case of comparing a two-stream reference to a single-stream hypothesis.

This dissertation will therefore address two gaps. The first is the absence of a reusable, reconfigurable synthetic mixture generation framework for overlapping speech that can manipulate overlap, noise, duration and speaker count as experimental variables. The second is the absence of an estimation algorithm for the particular evaluation scenario in which a potentially overlapping two-stream reference must be compared with a single-stream hypothesis transcript. The proposed solution is motivated by the second gap. It is to make the evaluation of mismatched reference and hypothesis structure more informative and fair. In conclusion, these problems define the niche of this dissertation: controlled robustness benchmarking for overlapping speech and an evaluation method for mismatched reference and hypothesis structures.

## 2.3 Recent Advances

### 2.3.1 Realism and Generalisation in Overlap Modelling

Recent work has attempted to reduce some of these limitations. Some research has focused on improving the realism of synthetic overlap. Yang et al. [14] criticise the standard practice of randomly mixing utterances and propose a more realistic simulation by learning overlap patterns from real conversation, and representing overlap patterns as a sequence of discrete tokens. Their results show that the ASR model trained with a realistic overlap simulation consistently outperforms models trained with random overlap. In the benchmarking direction, Cornell et al. [24] spurs the generalisation across speaker counts, formal vs informal settings, meeting duration, acoustic scenarios and recording configurations. Although they have made valuable progress from optimisation under specific conditions to broader generalisation, improving realism, they have yet to provide systematic frameworks to control overlap, SNR, duration, and speaker count in a clean design.

### 2.3.2 Integrated Pipelines and End-to-End ASR for Overlapping Speech

A second direction of work has concentrated on integrated meeting-recognition pipelines. [21] shows that an end-to-end modular system that integrates independently trained speech separation,

diarisation and transcription modules can achieve competitive performance on LibriCSS. Chen et al. [25] demonstrate that the combination of continuous speech separation (CSS) and conformer-based ASR can achieve state-of-the-art performance in both single-channel and multi-channel settings on LibriCSS. von Neumann et al. [17] use a pipeline that combines the CSS system with TF-GridNet separation and a speaker-agnostic ASR and achieves outstanding performance on ORC-WER. They also propose a syntactically informed diarisation for turn detection, which results in exceptional performance on cpWER. The strength of these modules is that they treat meeting transcription as an all-in-one problem rather than a pure ASR problem. This, conversely, brings the drawback of ambiguity on the source of improvements, which makes it harder to locate the specific factors that contribute to the improvements and also makes it harder to locate the point of failure.

A third line of work has turned to an end-to-end ASR framework for speaker-attributed modelling. Kanda et al. [26] present a transformer-based end-to-end model that performs speaker counting, speech recognition and speaker identification, reducing the dependence on separated separation and diarisation modules. Chang et al. [27] shows that such a model is susceptible to the mismatch between training and testing conditions and introduces a hypothesis stitcher that fuses multiple hypotheses into a single output to help improve accuracy. Kanda et al. [28] push this further by showing speaker-attributed transcription can be used to derive diarisation output, achieving a comparable diarisation error rate to existing diarisation methods and better if the number of speakers is not known. Conclusively, these studies are important because they treat lexical content and speaker identity as mutually informative independent tasks. On the flip side, they also reveal that since these systems are more specialised, the training assumptions are stronger, and the resulting outputs are less comparable to single-stream ASR baselines.

More recent papers have tried to reduce the gap between theoretical models and practical applications. Cui et al. [29] propose a pipeline for real-life applications that involves voice activity detection (VAD), speaker diarisation (SD), and speaker-attributed ASR (SA-ASR). They show that using VAD results to finetune the SA-ASR model significantly reduces the speaker error rate (SER) and demonstrate that extracting speaker embedding from SD can further improve the SER. Raj et al. [30] push the SURT framework from speaker-agnostic to speaker-attributed ASR by adding a speaker branch and prefixing each chunk with speaker information. Zheng et al. [31] introduces the DNCASR model for joint neural speaker clustering and ASR, effectively addresses overlapping speech and achieves strong cpWER performance on AMI. These studies indicate that the field is moving towards more application-oriented systems of long-form, speaker-consistent and meeting-level transcription. Nonetheless, they still rely on a small set of benchmarks. They therefore match the need for studies that evaluate ASR systems under different controlled conditions.

### 2.3.3 Unified Metrics for Multi-Speaker ASR Evaluation

Recent literature on ASR metrics provides evolving insights into multi-speaker evaluation. von Neumann et al. [17] contribute to standardising the computation of WER, cpWER, ORC-WER and MIMO-WER as a toolkit. von Neumann et al. [23] contributes to the conceptual framework for clarifying which metrics are suitable for which output structure and which aspect of performance. They also propose efficient approximation algorithms for expensive metrics like ORC-WER and DI-cpWER. These works discourage the default use of a single metric for outputs that embody different assumptions about speaker structure and timing. However, even this more mature scoring literature still treats the metrics primarily as ways of assessing matching systems. It does not directly solve the narrower experimental problem of how to compare two-stream references with single-stream hypotheses. That is, therefore, the space in which the present study makes its methodological contribution.

### 2.3.4 Summary

Overall, parallel research has advanced in three main directions. It has made the synthetic overlap more realistic rather than purely random. It has strengthened meeting transcription through integrated modular pipelines that combine separation, diarisation, and recognition. It has progressed ASR evaluation metrics towards a more comprehensive approach. However, it still leaves a gap in not having a lightweight, reusable framework for systematically varying degradation factors and the metrics that handle stream-mismatched evaluation. The present dissertation is designed to contribute to that missing middle ground.

# Chapter 3

# Architectural Design

## 3.1 System Overview

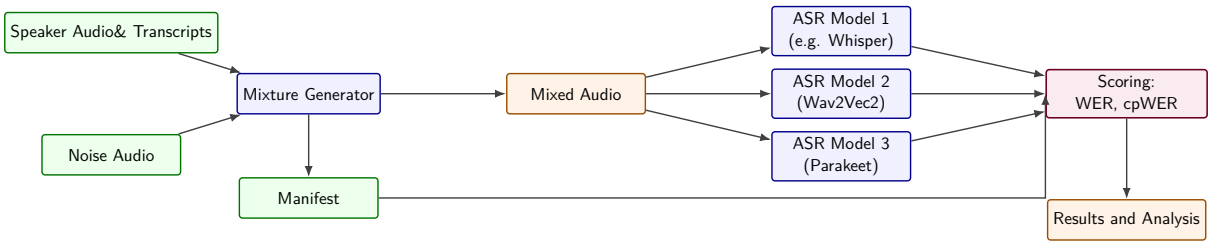


Figure 3.1: System data flow: input speech and noise files (.wav or .flac) are combined into mixtures, which are then transcribed by ASR models. The transcripts are scored against the reference with WER and overlap-aware metrics.

Figure 3.1 depicts the high-level data flow through the system. Inputs include clean speaker audio folders (containing audio of a single speaker, with speaker ID as the folder name and the corresponding transcripts) and noise audio folders (.wav or .flac, with noise type as the folder name). A YAML configuration file specifies parameters for mixture generation, including the number of mixtures, SNR levels (SNR), overlap ratios (ovr), duration, and speaker count (number of unique speakers). The *Mixture Generator* module creates synthetic mixtures based on these configuration and outputs both the audio files and a manifest (containing metadata and transcripts). The mixed audio is fed into the target ASR models for inference. The result hypotheses are scored against the reference transcripts using evaluation metrics. The resulting hypotheses are then evaluated against the reference transcripts using specified evaluation metrics. The final results is analysed and compiled into a final report.

## 3.2 Synthetic Mixture Generation

The mixture generation process comprises three steps: (1) **Speaker and clip selection**: randomly select the required number of speakers and clips from the clean speech dataset, (2) **Offset scheduling**: (1) **Offset computation**: calculate the time offsets for each utterance such that the overall overlap ratio approaches the target. (2) **Time offset adjustment**: adjust the time offsets to ensure the overall overlap ratio approaches the target. (3) **Noise addition**: add noise to the mixture at the specified SNR level. The output of this process will be a set of audio with different speaker counts and overlap ratios as base clips, and each base clip will have variants with different SNR levels. The metadata and transcripts of the mixtures are stored in a manifest file.

### 3.2.1 Speaker and Clip Selection

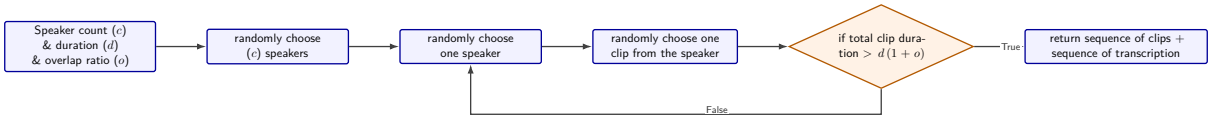


Figure 3.2: Speaker and Clip Selection Process

This process takes input of the target speaker count $c$ and duration constraints $d$ (minimum duration of final mixed audio), and returns: (1) a sequence of clips satisfying the target speaker count and duration constraints, (2) the corresponding sequence of transcripts.

Figure 3.2 shows the process that is designed to create a sequence of clips with specified speaker count and duration. A set of $c$ speakers is selected at uniform random without replacement. During the clip selection process, the next speaker is chosen with probability inversely proportional to the number of clips contributed from each speaker to balance the number of clips per speaker (with probability of 0 for speakers from the previous iteration). The random selection of clips is uniform random with replacement. The process iterates until the total duration of the selected clips exceeds the target duration adjusted by the overlap ratio (to account for the overlapping region that is double-counted here). The adjusted total clip duration $T_{sum}$ is calculated by:

$$
\begin{aligned}
\mathrm{OVR} &= T_{overlap}/d \\
T_{duration} &= T_{sum} - T_{overlap} \\
T_{overlap} &= T_{duration} - d \\
\mathrm{OVR} &= (T_{sum} - d)/d \\
\Rightarrow \quad T_{sum} &= d \cdot (1 + \mathrm{OVR})
\end{aligned}
$$

The output transcript is a sequence of speaker-attributed transcripts.

### 3.2.2 Offset Scheduling

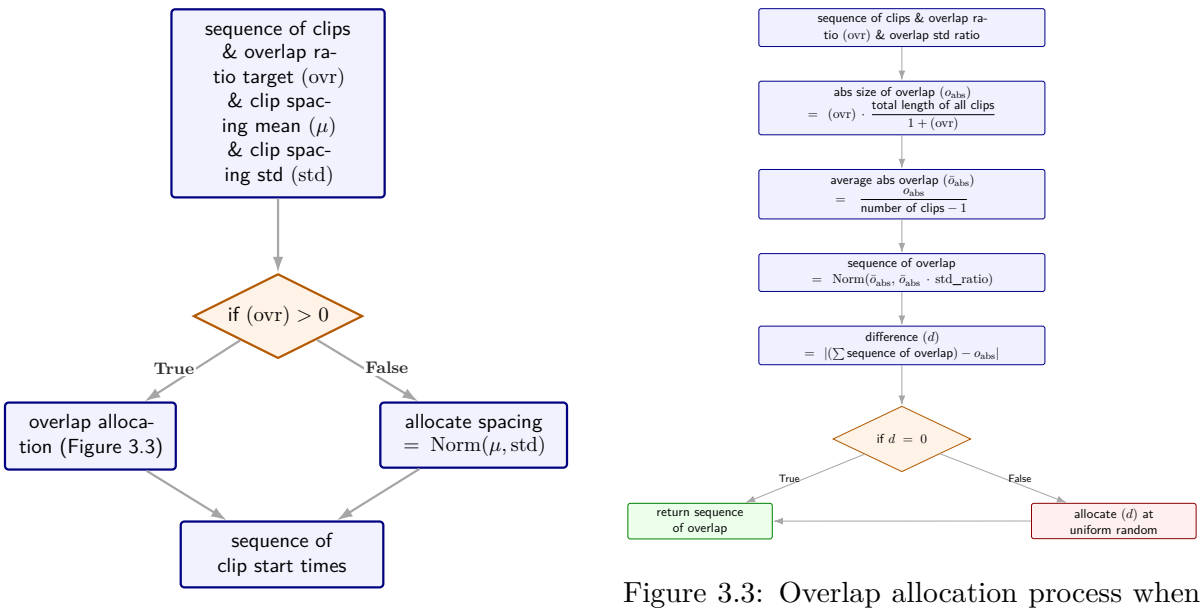


Figure 3.3: Overlap allocation process when overlap ratio $> 0$

Figure 3.4: Offset Scheduling Process

This process takes input of a sequence of clips, overlap ratio (ovr $\in [0, 1]$), overlap std ratio, and clip spacing mean and std (for non-overlapping case) and outputs the start sample index for each clip where the overlap ratio is exactly as specified.

In the case where the overlap ratio is zero, the start time of each clips is determined by allocating a spacing that is a random sample from a normal distribution $\texttt{Norm}(\mu, \text{std})$ where $\mu$ and std are the mean and std of the clip spacing specified. In the case where overlap ratio is greater than zero, we first compute the target overlap $o_{\text{abs}}$ in samples by:

$$o_{\text{abs}} = \frac{\text{ovr}}{1 + \text{ovr}} \cdot \sum \text{clip duration}.$$

and, for $n$ number of clips, the average overlap $\bar{o}_{\text{abs}}$ by $\bar{o}_{\text{abs}} = \frac{o_{\text{abs}}}{n-1}$. Then each overlap between adjacent clips is then sampled from a normal distribution $\texttt{Norm}(\bar{o}_{\text{abs}}, \bar{o}_{\text{abs}} \cdot \text{std_ratio})$ where the overlap is capped by the duration of the preceding and succeeding clips. After that, a residual correction is applied so that the sum of sampled overlaps equals the target overlap exactly. The difference diff $= o_{\text{abs}} - \sum$ sampled overlaps. If the difference is greater than zero, we randomly select overlaps that are smaller than the preceding clip and succeeding clip duration, and add one sample to the selected overlaps until the difference is zero. If the difference is smaller than zero, we randomly select overlaps that are greater than zero and subtract one sample until the difference is zero. Finally, the start sample index of each clip is determined by adding the spacing (overlap will be treated as negative spacing). The start and end timestamp of each clip is then put into the manifest.

### 3.2.3 Noise Addition

The noise addition process takes input of the mixed audio, set of noise audio, noise type and target SNR level ($\text{SNR}_{dB} \in \mathbb{R}$ or clean, measured over active speech time), and output the noisy mixed audio, where same base audio can produce different noisy versions. The noise type can be domestic (D), public (P), transportation (T), nature (N), office (O), street (S) or all (A) (more details in Section 6.1). Then sample noise audio from the specified noise type(s) uniformly at random with replacement, and concatenate the noise audio until the total length is reached (exceeding part will be truncated). Finally, the noise is scaled by a factor $\alpha$ to meet the target SNR level. The scaling factor $\alpha$ is computed by:

$$\alpha = \frac{A_{\text{active speech}}}{A_{\text{noise}} \times 10^{\text{SNR}_{dB}/20}}.$$

where $A_{\text{active speech}}$ is the root mean square (RMS) amplitude of the active speech region in the mixture, and $A_{\text{noise}}$ is the RMS amplitude of the noise segment before scaling. The active speech region is determined by the RMS of all clips in the mixture. The SNR is measured relative to the active speech region to ensure the noise level is accurately proportional to the speech content. Finally, the scaled noise is added to the mixture to produce the final audio.

This algorithm is designed to support controlled experiments on overlap ratio, SNR level, speaker count, duration and noise type by varying each factor independently. It also supports noise analysis on the same clip with different noise types and SNR levels.

### 3.2.4 Manifest Structure

The manifest is a csv file that contains the metadata and transcripts of the mixtures.

| Field | Description | Example |
|---|---|---|
| clip_id | Unique identifier for each generated mixture. | mix_0000003_0.00_2_7.4_D |
| audio_path | Relative path to the generated mixed waveform file. | Output/audio/...wav |
| transcript | Serialized list of speaker-attributed utterances with start and end timestamps. | [(spk,text,t0,t1), ...] |
| overlap_ratio_target | Target overlap ratio specified by the generation configuration. | 0.0 |
| overlap_ratio_actual | Overlap ratio measured from the generated mixture. | 0.0 |
| speakers_count | Number of unique speakers. | 2 |
| snr_db | Target signal-to-noise ratio in dB; can be empty for clean/no-noise settings. | 7.4, 0.0, -5.0 |
| noise_type | Noise category code used in sampling (e.g., Domestic/Public/Transport). | D, P, T |
| overlap_mask_path | Path to overlap mask data or placeholder if not generated. | no mask |
| source_files | Serialized list of source speech files used to construct the mixture. | ['/Data/LibriSpeech/train-clean-100/8123/275209/8123-275209-0011.flac', ...] |
| noise_files | Serialized list of selected noise files (empty for clean conditions). | 'Data/Noise/TBUS/ch06.wav' |

Table 3.1: Manifest schema used to store synthetic mixture metadata and transcripts.

# Chapter 4

# System and Data Flow

Figure 4.1 shows the high-level data flow of the system. The inputs are folders of **speaker audio** (clean speech WAVs with transcripts) and **noise audio** (e.g. environmental sounds). A configuration YAML file specifies parameters (target overlap ratios, SNR values, clip length, speaker count, etc.). The *Mixture Generator* module loads all audio files and transcripts, and repeatedly constructs mixtures using the `generate_audio.py` script. Each mixture is passed to ASR *Evaluation* (the `asr_helper.py` pipeline), which runs several models to produce hypotheses. The *Metrics* module (using `wer_helper.py`) computes WER, cpWER, DI-cpWER, etc. for each transcription. Finally, results (scores and logs) are collated for analysis.

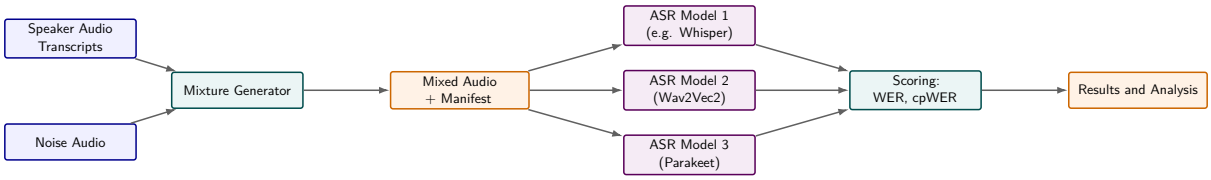


Figure 4.1: System data flow: input speech and noise are combined into mixtures, which are then transcribed by multiple ASR models; transcripts are scored with WER and overlap-aware metrics.

In more detail, the mixture generation component comprises three steps: (1) **Selecting speakers and clips**: randomly choose $n$ speakers and their utterances; (2) **Offset scheduling**: compute time offsets to position each utterance so that the overall overlap ratio approaches the target; (3) **Noise addition**: mix in noise at a specified SNR. These steps yield a single mixed waveform plus metadata (transcripts aligned to mixture times). Table 4.1 lists the main software tools and libraries used.

## 4.1 Mixture Generation Algorithm

The core mixture generation algorithm is implemented in `generate_audio.py`. First, the script loads all clean speech files (grouped by speaker) and noise files. A transcript map is built from available transcripts (e.g. LibriSpeech or VCTK labels) keyed by utterance ID.

The main function `build_base_mixture` selects $n$ speakers and samples utterances (potentially concatenating multiple segments per speaker to reach minimum length). It then calls `generate_offsets` to compute start times:

This function attempts to align the chosen utterances so that the fraction of overlapping samples matches `overlap_ratio_target`. It does so by random sampling of overlap extents and iterative adjustment. In the no-overlap case, it simply spaces speakers apart by a random gap.

Once offsets are determined, `build_base_mixture` sums the waveforms at those offsets:

```python
def generate_offsets(
    audios: List[np.ndarray],
    n_speakers: int,
    overlap_ratio_target: float,
    sample_rate: int,
    speaker_spacing: tuple,
    sample_length: int,
) -> Tuple[List[int], float, np.ndarray, int]:
    offsets = [0]
    p = len(audios[0])
    active = np.zeros(sample_length, dtype=np.int32)
    active[: len(audios[0])] += 1

    if n_speakers == 1 or overlap_ratio_target <= 0:
        for a in audios[1:]:
            p += max(0, np.random.normal(*speaker_spacing) * sample_rate)
            offsets.append(int(p))
            active[int(p): int(p) + len(a)] += 1
            p += len(a)
        return offsets, overlap_ratio_target, active, len(active)

    # ... otherwise allocate overlapping offsets ...
```

```python
mixture = np.zeros(sample_length, dtype=np.float32)

for i, (wave, off) in enumerate(zip(audio_parts, offsets)):
    if off + len(wave) > len(mixture):
        mixture = np.pad(mixture, (0, off + len(wave) - len(mixture)), "constant")

    # Record transcript segment timing
    speaker, text = transcript_parts[i][:2]
    transcript_parts[i] = (
        speaker,
        text,
        off / sample_rate,
        (off + len(wave)) / sample_rate,
    )

    # Add waveform
    mixture[off: off + len(wave)] += wave

rms_val = rms(np.concatenate(audio_parts))
return BaseMixture(
    wave=mixture,
    overlap_mask=active,
    source_files=chosen,
    transcript=transcript_parts,
    overlap_ratio_actual=actual_or,
    rms=rms_val,
)
```

Table 4.1: Main tools and libraries in the system.

| Component / Library | Purpose |
|---|---|
| Python 3.x | Implementation language |
| NumPy, SciPy | Numerical computing, resampling |
| SoundFile (PySoundFile) | Audio I/O (WAV/FLAC) |
| Librosa | Audio processing (resampling, padding) |
| Pandas | Metadata (CSV manifest) |
| PyTorch | Model inference engine (ASR models) |
| NVIDIA NeMo | ASR and audio processing toolkit |
| Hugging Face Transformers | Faster-Whisper, Wav2Vec2 pipelines |
| WhisperX | Whisper with timestamp alignment |
| FunASR | End-to-end ASR models (Nano-T, etc.) |
| Jupyter / CLI | Experiment control |

This produces a `BaseMixture` object containing the mixed audio and metadata (actual overlap, RMS value, transcripts with time). Finally, in the main loop the code iterates over conditions (overlap ratios, max speakers, SNR values, noise types) and generates final mixtures by adding noise:

```
noise = []

# Concatenate random noise clips until mixture length is reached
while len(noise) < len(mixture):
    npath = random.choice(noise_files[noise_type])
    tnoise, nsr = sf.read(npath)
    tnoise = resample(tnoise, nsr, sample_rate)
    noise = np.concatenate([noise, tnoise.astype(np.float32)])

noise = noise[: len(mixture)]
mixture_noisy = add_noise_for_snr(mixture, noise, snr_db, base_rms)
sf.write(out_audio / f"{clip_id}.wav", mixture_noisy, sample_rate)

rows.append(
    MixtureMeta(
        ...,  # other fields
        transcript=base.transcript,
        snr_db=snr_db,
        noise_type=noise_type,
        source_files=base.source_files,
        noise_files=noise_paths,
    )
)
```

Here `add_noise_for_snr` (from `generate_audio.py`) scales the noise to achieve the target SNR:

By saving each noisy mixture and its metadata (clip ID, transcripts, overlap, noise paths, etc.) to a CSV manifest, the system ensures reproducibility of every parameter.

```
def add_noise_for_snr(speech, noise, snr_db, base_rms):
    noise_rms = rms(noise)
    alpha = base_rms / (noise_rms * (10 ** (snr_db / 20)) + EPSILON)
    return speech + alpha * noise
```

## 4.2 Representative ASR Models

We evaluate several pre-trained ASR models to cover different architectures and training regimes. Table 4.2 compares their key properties. The selection includes large sequence-to-sequence models and smaller streaming ones:

- **Whisper (large-v3)** – an encoder-decoder model trained on multilingual weakly supervised data ; known for robustness to noise and disfluencies.
- **Wav2Vec2** – a self-supervised convolutional-Transformer model (e.g. base or large from Facebook) , fine-tuned on English data.
- **Parakeet-RNNT** – an NVIDIA NeMo multilingual RNN-Transducer (1.1B) model trained on extensive data, used for robust transcription and language identification.
- **WhisperX** – a variant of Whisper with timestamp refinement (for forced alignment of transcripts).
- **FunASR** – lightweight end-to-end ASR models (e.g. Nano-T) from the FunASR toolkit, enabling fast decoding.

We justify these choices as follows: Whisper and Wav2Vec2 are leading open-source ASR models, representing large-scale and self-supervised approaches respectively . NVIDIA's Parakeet RNN-T provides a state-of-the-art industrial model for multilingual ASR. WhisperX and FunASR add variety in capabilities (alignment, low-latency models). Each model has published results attesting to competitive WER performance under noise (e.g. Whisper and Wav2Vec2 ). We do not use proprietary services, focusing on open or academic systems.

Table 4.2: ASR models evaluated on the synthetic mixtures.

| Model | Type | Comments |
|---|---|---|
| Whisper (large-v3) | Seq2Seq Transformer | Multilingual, robust to noise |
| Wav2Vec2 (base/large) | Transformer (CTC) | Self-supervised, fine-tuned on LibriSpeech |
| Parakeet-RNNT | RNN-Transducer | Large (1.1B) multilingual model (NVIDIA NeMo) |
| WhisperX | Seq2Seq + alignment | Whisper with improved timestamping |
| FunASR (Nano-T) | Conformer (CTC) | Small model for low-latency ASR |

Each model is run on every generated audio file via `asr_helper.py`. The outputs are collected into a `MixtureTranscription` structure (mapping clip IDs and speaker IDs to hypothesis text). This uniform format simplifies downstream scoring. No fine-tuning is performed; we evaluate the zero-shot performance of these off-the-shelf models on the synthetic data, and also test their transfer to real recordings (using a reserved real multi-speaker corpus).

## 4.3 Evaluation Metrics

We use standard and overlap-aware metrics to quantify transcription accuracy. The basic measure is *Word Error Rate (WER)*, the Levenshtein distance normalized by reference length.

For multi-speaker outputs, we compute *concatenated minimum-permutation WER (cpWER)* : all utterances of each speaker are concatenated, and the speaker labels are optimally permuted to minimize total WER . This was introduced for CHiME 2020 and captures both ASR and diarization errors . We implement cpWER as in CHiME and Meeteval, and also compute a *diarization-informed cpWER (DI-cpWER)* by penalizing misaligned word-timestamps (as in our code's `DIcpWER` function). Additionally, we record *Character Error Rate (CER)* and *Insertion-Deletion-Substitution rates* for completeness. We do not use perceptual metrics (e.g. PESQ) as we focus on transcription. All metrics are averaged across mixtures with the same conditions to analyze trends.

# Chapter 5

# System Architecture and Implementation

The system is implemented in Python and consists of three stages: *mixture generation*, *ASR decoding*, and *metric computation*. Figure 4.1 shows the high-level data flow. First, input audio files (clean speech from each speaker, and optional background noise) are read. The `generate_audio.py` script computes random time offsets for each speaker to achieve a target overlap ratio, then sums the signals and noise at those offsets to create the synthetic mixture. The synthetic waveform and reference transcripts (with speaker labels) are then passed to the ASR stage.

### 5.0.1 Libraries and Tools

Key software includes Python 3.8+, with `numpy`, `soundfile/librosa` for audio I/O, `pandas` for metadata tables, and `torch/transformers` for ASR models. We also use the `whisperx` package for OpenAI Whisper models. The baseline ASR systems are representative neural models: wav2vec 2.0 [18], Whisper [19], and Conformer [**?** ]. Table 5.1 summarizes their architectures.

## 5.1 Synthetic Mixture Generation

The `generate_audio.py` script creates controlled multi-speaker mixtures. After loading individual speaker waveforms, it calls `generate_offsets` to determine start times. For example, the first speaker is placed at $t = 0$; subsequent speakers are placed at random delays sampled from a normal distribution. The code snippet below (simplified) shows this offset scheduling:

Listing 5.1: Offset scheduling algorithm (simplified).

```
def generate_offsets(audios: List[np.ndarray], n_speakers: int,
                     overlap_ratio_target: float, sample_rate: int,
                     speaker_spacing: tuple, sample_length: int):
    offsets = [0]
    p = len(audios[0])
    active = np.zeros(sample_length, dtype=np.int32)
    active[:len(audios[0])] += 1

    if n_speakers == 1 or overlap_ratio_target <= 0:
        for a in audios[1:]:
            p += max(0, np.random.normal(*speaker_spacing) * sample_rate)
            offsets.append(int(p))
            active[int(p):int(p)+len(a)] += 1
```

| Model | Architecture | Pretraining / Data |
|---|---|---|
| Wav2Vec 2.0 [18] | CNN + Transformer (self-supervised) | Pretrained on LibriSpeech (960 h) |
| Whisper [19] | Encoder-decoder Transformer | Trained on 680 k hours of diverse audio |
| Conformer [? ] | Convolution-augmented Transformer | Trained on Librispeech (10 k h) |

Table 5.1: ASR models evaluated.

```
        p += len(a)
    return (offsets, overlap_ratio_target, active, len(active))
# Otherwise, allocate overlap duration across speakers (omitted)
```

Each speaker signal is added to the mixture at its offset, and noise is scaled to reach the specified SNR. Metadata (speaker IDs, offsets, transcripts) is recorded in a Pandas DataFrame (see `helper_class.py`).

## 5.2 ASR Decoding

In `asr_helper.py`, each mixture is passed to selected ASR models. HuggingFace models use `pipeline("automatic-speech-recognition")`. Whisper models use `whisperx.load_model`. The output transcripts may include special tokens marking speaker changes (e.g. serialized output). We parse each hypothesis into (speaker, text) pairs for scoring.

## 5.3 Error Metrics: WER, cpWER, DI-cpWER

Standard word error rate (WER) is computed per speaker by aligning hypothesis vs. reference words. For overlapping speech, we implement *concatenated-permutation WER* (cpWER) as defined by von Neumann et al. [23]. This finds the best mapping of hypothesis speakers to reference speakers to minimize total errors. The code below sketches the calculation; setting `DI=True` computes the DI-cpWER variant (ignoring speaker labels):

Listing 5.2: Computing cpWER (with optional DI mode).

```
def cpWER(ref: List[tuple[str, str]], hyp: List[tuple[str, str]], DI=False) -> f
    # Convert (speaker, text) lists to dict of full utterances
    ref_dict = transcript_to_dict(ref)
    hyp_dict = transcript_to_dict(hyp)
    for spk in ref_dict:
        ref_dict[spk] = ' '.join(ref_dict[spk])
    for spk in hyp_dict:
        hyp_dict[spk] = ' '.join(hyp_dict[spk])
    # If DI=True, ignore speaker IDs (diarization-invariant)
    # Find optimal assignment (permutation search or greedy)
    ...
```

The `wer_evaluation.py` script computes and reports WER, cpWER, and DI-cpWER for each evaluation set.

# Chapter 6

# Datasets, ASR Models and Preprocessing Strategies

## 6.1 Datasets

The evaluation will be conducted on the following datasets:

- **Single Speaker Clean Speech (for synthetic mixtures)**
  - **LibriSpeech train-clean-360** LibriSpeech is a corpus of approximately 1000 hours of 16kHz read English speech, prepared by Vassil Panayotov with the assistance of Daniel Povey. The data is derived from read audiobooks from the LibriVox project, and has been carefully segmented and aligned. [32]
  - **CSTR VCTK Corpus** [33]: a large-scale speaker recognition dataset containing speech from thousands of speakers, suitable for creating synthetic mixtures with varying overlap and SNR.
- **Real-world Overlapped Speech**
  - **CHiME6_eval** [2]: English multi-channel far field meeting data used in the CHiME-6 Challenge. It is derived from CHiME5 by fixing some array synchronization errors.
  - **The AMI Corpus** [1]: The AMI Meeting Corpus is a multi-modal data set consisting of 100 hours of meeting recordings.
- **Noise Data**
  - **DEMAND** [34]: The DEMAND (Diverse Environments Multichannel Acoustic Noise Database) aims to provide a set of recordings that allow testing of algorithms using real-world noise in a variety of settings.

## 6.2 Baseline Models under Evaluation

The evaluation will include the following ASR models:

- **Faster Whisper** (OpenAI): faster-whisper is a reimplementation of OpenAI's Whisper model using CTranslate2, which is a fast inference engine for Transformer models. [35]
- **Wav2Vec2-Large-960h** (Meta AI): The large model pretrained and fine-tuned on 960 hours of Librispeech on 16kHz sampled speech audio. When using the model make sure that your speech input is also sampled at 16Khz. [36]

- **parakeet-tdt-0.6b-v3** (NVIDIA): parakeet-tdt-0.6b-v3 is a 600-million-parameter multilingual automatic speech recognition (ASR) model designed for high-throughput speech-to-text transcription. [37]
- **NVIDIA NeMo Canary-Qwen-2.5B** (NVIDIA): NVIDIA NeMo Canary-Qwen-2.5B is an English speech recognition model that achieves state-of-the art performance on multiple English speech benchmarks. [38]
- **granite-speech-3.3-8b** (IBM Research): Granite-speech-3.3-8b is a compact and efficient speech-language model, specifically designed for automatic speech recognition (ASR) and automatic speech translation (AST). [39]
- **Phi-4-multimodal-instruct** (Microsoft): Phi-4-multimodal-instruct is a lightweight open multimodal foundation model that leverages the language, vision, and speech research and datasets used for Phi-3.5 and 4.0 models. [40]

## 6.3 Preprocessing Strategies

The evaluation will investigate the following preprocessing strategies to assess their impact on ASR performance under overlapping speech and noise conditions:

- **ESPnet Single Channel Speech Enhancement** (ESPnet-SE) [41]
- **ESPnet Speech Separation** (ESPnet-SS) [41]
- **SepRefomer** [42]
- **VoiceFilter** [43]

# Chapter 7

# Experimental Design

## 7.1 Objective and Scope

This model evaluation plan evaluate the robustness of multiple English automatic speech recognition (ASR) transcription models under (i) overlapping speech and (ii) additive noise, and investigates and quantifies how different preprocessing strategies affect performance across these conditions and different ASR models. The plan follows: compare ASR performance against overlap ratio and SNR, evaluate preprocessing impacts (normalisation, enhancement, separation, and target-speaker filtering), and an study where speaker embedding are provided vs not provided. The evaluation will be conducted under a controlled experimental design, with systematic variation of overlap ratios, SNR levels, and preprocessing, and will be statistically analysed to determine the significance of observed differences in ASR performance. The results will inform the best pipelines for improving ASR robustness in real-world scenarios with overlapping speech and noise.

## 7.2 Research Questions

**RQ1:** How does ASR performances change as the *overlap ratio* increases, under constants SNR levels, and how does this vary across different ASR models?

**RQ2:** How does ASR performance change as *SNR levels* increase, under constant overlap ratios, and how does this vary across different ASR models?

**RQ3:** Do trends observed on *synthetic mixtures* in RQ1 and RQ2 transfer to real-world audio conditions, and how do they differ across ASR models?

**RQ4:** How do different preprocessing models affect ASR performance under overlapping speech and noise conditions?

**RQ5:** What is the impact of providing *speaker embeddings* on ASR performance under overlapping speech conditions?

## 7.3 Experimental Design Overview

The evaluation is performed under a controlled crossed design to quanitfy the effects of (i) overlap ratio, (ii) number of overlapping speakers, (iii) Singal-to-Noise Ratio (SNR), (iv) noise type, (iv) preprocessing strategy, and (v) availability of speaker embedding information. The design includes two types of audio data:

1. **Synthetic mixtures (controlled):** audio mixtures are generated with precise control of overlap ratio, number of concurrent speaker, SNR, and noise type. This enables causal attribution of performance changese to specific factors and their relationships.

2. **Real-world overlapped speech (ecological validity):** evaluation is implemented on real-world conversational datasets to assess the transferability of the trends observed on synthetic mixtures to real-world conditions.

Across both tracks, all models are evaluated on identical audio segments and references to enable paired statistical comparisons.

### 7.3.1 Factors and Levels

Table 7.1 summarises the main experimental factors. The full condition matrix is implemented via a fractional factorial design to keep the total number of runs computationally feasible while preserving coverage of key interactions (e.g., overlap × SNR, preprocessing × overlap, model × preprocessing).

Table 7.1: Experimental factors and planned levels.

| **Factor** | **Levels** |
|---|---|
| Overlap ratios (OVRs) | $0\%, 14\%, 20\%, 40\%$ |
| Max concurrent speakers ($S$) | 2 |
| SNR (dB) | clean, 10, 0, $-5$ |
| Noise type | domestic, public, transportation |
| Audio duration | Synthetic: $0\%, 14\%, 20\%, 40\%$ 120-140s, Real-world: long-form meetings |
| Preprocessing pipeline | baseline, enhancement, separation, target-speaker filtering and combination of them |
| Speaker embeddings | provided vs not provided (for target-speaker methods only) |

#### Overlap Ratios Choice

I have chosen 0%, 14%, 20%, 40%.

- 0% is chosen to be the control where it represent completely clean audio in which there is no overlapping.
- 14% is chosen to represent the mean overlap ratio for meetings. The percentage of overlapped words (excluding backchannels like "uh-huh") in meetings is 14.1% [44].
- 20% is chosen to represent the maximum overlap ratio in a natural meeting or casual dinner parties [3] [12].
- 40% is chosen as it is supported by the LibriMix benchmark, which chooses 40% as the maximum overlap ratio. This is chosen to stress-test under high-overlap conditions [3].

#### Maximum Concurrent Speakers Choice

Maximum concurrent speaker is chosen to be 2 as more than 2 people speaking simultaneously is very rare in real meets [45].

**Signal-to-noise Raio Choice**

I have chosen *clean*, 7.4dB, 0dB, -5 db

- *clean* is chosen to be the control where it represent clean background with no noise.
- 7.4dB is chosen to represent the noisy environment as it is the mean SNR for noisy environment [46].
- 0db is chosen to represent environment where the noise and speech are equally loud, which is a challenging but realistic condition in very noisy environments.
- -5db is chosen to represent environment where the noise is louder than speech in which even human listeners struggle to understand speech, providing a stress-test condition for ASR.

**Noise Type Choice**

I have chosen three noise types: domestic, public, and transportation. These types represent different real-world scenarios with varying noise characteristics.

- **Domestic noise** includes kitchen noise when food is being prepared, living room noise and washing room with front-loading washing machine. These noises are commonly found in home environments that a home-based voice assistant would encounter.
- **Public noise** includes noise from main transfer area of subway station, office cafeteria, and university restaurants during lunchtime. These noise represent common public environments where ASR like mobile devices may be used in.
- **Transportation noise** includes noises in subway, public bus and private passenger vehicle. These noises represent common transportation environments where ASR like mobile devices may be used in.

### 7.3.2 Performance Metrics

The performance of each experimental unit is measured using standard automatic speech recognition (ASR) metrics, including:

- **Word Error Rate (WER)**: The percentage of words that are incorrectly transcribed relative to the reference transcript.
- **Concatenated minimum-Permutation WER (cpWER)**: It assumes that the system groups its output by speakers and the recognized transcripts, i.e., the concatenation of several utterances, of each speaker are compared to the reference transcripts of each speaker.
- **Diarization-Invariant cpWER (DI-cpWER)**: It compute cpWER under the assumption of all speaker labels are correct. This is to evaluate the diarization-invariant performance of ASR models. [7]

### 7.3.3 Experimental Units

The experimental unity is an pairing of an audio clip (with specific overlap, SNR, noise type), a preprocessing pipeline (including no preprocessing), target-speaker embedding availability (for target-speaker methods), and an ASR model.

$$\text{clip} \rightarrow \{\text{preprocessing pipeline}\} \times (\text{speaker embedding}) \times \{\text{ASR model}\}$$

Each unit is evaluated to produce a set of performance metrics (e.g., WER, overlap-specific WER) that are then statistically analysed to answer the research questions.

## 7.4 Datasets and Data Splits

### 7.4.1 Track A: Synthetic Mixture Dataset (Controlled Overlap)

#### Source Speech Selection

Synthetic mixtures are constructed from clean English speech utterances with high-quality transcripts with accurate timestamps. The source speech is selected from datasets such as LibriSpeech and VCTK, which provide a large number of speakers and utterances to enable the generation of mixtures with varying overlap ratios and speaker combinations.

#### Overlap Construction

For each mixture, a target overlap ratio (OVR) is achieved by controlling the alignment of selected utterances. The overlap ratio is defined as:

$$\mathrm{OVR} = \frac{\text{time with} \geq 2 \text{ active speakers}}{\text{active speech time (time with at least one speaker)}}.$$

In the controlled design, the maximum number of concurrent speakers is fixed to $K = 2$ to isolate the effect of overlap ratio. During generation, an active mask $a(t)$ is produce indicating the number of speaker in any given moment. The actual overlap ratio is then calculated by

$$\mathrm{OVR}_{\text{actual}} = \frac{\sum_t 1[a(t) > 1]}{\sum_t 1[a(t) \geq 1]}$$

#### Noise Addition and SNR Control

Additive noise is added to the clean mixtures to achieve target SNR levels. The SNR (RMS) is calculated as:

$$\mathrm{SNR}_{dB} = 20 \log_{10} \left( \frac{\mathrm{A}_{\text{active speech}}}{\mathrm{A}_{\text{noise}}} \right),$$

where $A$ denotes root mean square (RMS) amplitude. $A_{\text{active speech}}$ denotes the RMS amplitude of active segments such that the SNR level is silence invariant and closer to human perception. The noise is first normalised and rescaled by $\alpha$ to achieve the target SNR (Table 7.1).

$$\alpha = \frac{A_{\text{active speech}}}{A_{\text{noise}} \times 10^{\mathrm{SNR}_{dB}/20}}.$$

Type of noise is varied across conditions, with noise segments selected from dataset DEMAND to represent a range of real-world environments.

### 7.4.2 Track B: Real Overlapped Speech Dataset

A real conversational or meeting dataset containing natural overlap is used to validate transferability of findings. Real audio is segmented into clips comparable in duration to the synthetic evaluation. Where available, target-speaker information (e.g., from diarisation) is used for the target-speaker embedding conditions.

# Chapter 8

# Preprocessing Pipelines

## 8.1 Preprocessing Structure

### 8.1.1 Pipeline Definition

Each preprocessing method is implemented as a deterministic function $f(\cdot)$ applied to waveform audio, producing a processed waveform:

$$x' = f(x; \theta),$$

where $\theta$ are fixed parameters. Pipelines are defined as ordered compositions (e.g., normalisation $\rightarrow$ enhancement $\rightarrow$ ASR), where normalisation is always applied as it is computationally cheap and necessary for consistent input to downstream models, but to avoid combinatorial explosion, only a limited set of combinations is evaluated.

### 8.1.2 Experimental Plan

The experimental study includes:

1. **Individual preprocessing methods:** baseline vs each preprocessing method in isolation.
2. **Selected multi-step pipelines:** a small set of combinations chosen based on theoretical complementarity.
3. **Speaker embedding experiment:** for target-speaker filtering, evaluate *with* vs *without* embeddings, holding all other factors fixed.

If separation methods are constrained to $K = 2$ due to model limitations, this is explicitly reported and separation results are presented as a focused subset.

## 8.2 ASR Inference Controls

To ensure fair comparisons, inference settings are held constant per model:

- **Sampling rate:** audio is resampled to each model' s required input rate (typically 16 kHz), with consistent resampling across conditions.
- **Decoding configuration:** beam size / temperature / language constraints are fixed and documented for each model.
- **Chunking/segmentation:** if models require chunked inference, chunk length and overlap are fixed and applied identically across conditions.

All model versions, checkpoints, and library commits are recorded for reproducibility.

## 8.3 Evaluation Protocol

### 8.3.1 Reference Transcripts

For synthetic mixtures, the reference includes the known transcript for each source speaker. For real datasets, references are taken from official annotations. Where multi-speaker references exist, meeting-style scoring is used.

### 8.3.2 Primary and Secondary Metrics

**Primary outcome:** concatenated permutation word error rate (cpWER) on the full clip reference(s).
**Secondary outcomes:**

- WER on clips with no overlaps and non-overlapped regions (for clips with OVR $> 0\%$);
- DI-cpWER for all clips with OVR $> 0\%$ to evaluate ASR performance independent of diarization errors;

### 8.3.3 Overlap-aware Evaluation

To directly answer overlap-focus questions, overlap-aware evaluation is performed by cpWER and by restricting scoring to words occurring within overlapped regions (using the overlap mask for synthetic mixtures, and overlap annotations/estimates for real audio). This yields **overlap-only WER**, which isolates performance under simultaneous speech from non-overlapped regions.

### 8.3.4 cpWER for Multi-speaker References

When multi-speaker references are available and speaker attribution is not guaranteed in the hypothesis, meeting-style scoring such as concatenated permutation WER (cpWER) is used to reduce sensitivity to speaker ordering. [7]

# Chapter 9

# Evaluation Metrics

## 9.1 Overview and Principles

All statistical analysis is performed at the *clip level*, where each clip is evaluated under multiple paired conditions (e.g., preprocessing A vs B) on the *same* audio content. This paired design reduces variance and enables stronger inference about the effect of overlap, SNR, preprocessing, and model choice. :contentReference[oaicite:1]index=1

Unless otherwise stated, all results are reported as mean ± 95% confidence interval (CI) computed via bootstrap resampling over clips within a condition (Section 9.3).

## 9.2 Response Variables

For each experimental unit (clip × preprocessing × embedding setting × ASR model), the following response variables are computed:

- $y_{WER}$: Word Error Rate (WER) over the full clip reference.
- $y_{oWER}$: overlap-only WER, computed only on words that occur within overlapped regions.
- $y_{cpWER}$: cpWER for multi-speaker reference scoring when applicable.

Overlap-only scoring uses the overlap mask stored in synthetic mixtures, and overlap annotations/estimates for real audio (Chapter 3–4). :contentReference[oaicite:2]index=2

## 9.3 Bootstrap Confidence Intervals

For each condition cell (defined by overlap ratio, $K$, SNR, noise type, preprocessing, and ASR model), a stratified bootstrap over clips is used to estimate uncertainty. Let $\{y_i\}_{i=1}^{n}$ be clip-level scores within the cell. We sample with replacement $n$ clips for $B$ resamples (e.g., $B = 10{,}000$), compute the mean score for each resample, and take the 2.5th and 97.5th percentiles to form a 95% CI.

## 9.4 Paired Comparisons (Preprocessing Effects)

To answer RQ4 and RQ5, preprocessing pipelines are compared *paired* on identical clips:

$$d_i = y_i^{(A)} - y_i^{(B)},$$

where $y_i^{(A)}$ and $y_i^{(B)}$ are clip-level scores under pipelines A and B, respectively. We report:

- mean difference $\bar{d}$ and 95% CI via paired bootstrap;
- a non-parametric paired test (Wilcoxon signed-rank) as a robustness check.

Because multiple preprocessing comparisons are performed across many conditions and models, $p$-values are adjusted using Holm–Bonferroni correction.

## 9.5 Main Effects and Interactions (Model-Based Analysis)

To quantify interactions such as preprocessing × overlap ratio and model × overlap ratio, a regression-style analysis is applied on clip-level scores. Let the design factors be: overlap ratio (OR), max concurrent speakers ($K$), SNR, noise type, preprocessing pipeline, ASR model, and speaker embedding availability (when relevant). The analysis focuses on:

- **main effects**: OR, $K$, SNR, preprocessing, model;
- **key interactions**: OR×preprocessing, OR×model, preprocessing×model.

A mixed-effects model is used when feasible to account for repeated measurements per clip:

$$y_i = \beta_0 + \beta^\top X_i + u_{\text{clip}(i)} + \epsilon_i,$$

where $X_i$ contains fixed effects and interaction terms, $u_{\text{clip}(i)}$ is a random intercept for clip, and $\epsilon_i$ is residual error.

If mixed-effects estimation is unstable for WER distributions, the primary inference falls back to: (i) paired bootstrap comparisons for effect sizes, and (ii) condition-wise aggregated analysis with robust standard errors.

## 9.6 Reporting of Effect Sizes

Beyond statistical significance, the following are reported:

- absolute WER change: $\Delta WER$;
- relative WER reduction: $(WER_B - WER_A)/WER_B$;
- overlap-only improvements $\Delta oWER$ as the primary overlap-focused outcome.

## 9.7 Transferability: Synthetic vs Real (RQ3)

To assess transferability, model and preprocessing rankings (by WER and overlap-only WER) are computed on synthetic mixtures and real datasets separately. Agreement is quantified using rank correlation (Spearman) across systems and key operating points (e.g., OR $\geq$ 20%, SNR $\leq$ 0 dB).

# Chapter 10

# Statistical Analysis Plan

All statistical analyses are performed at the *clip level*, where each clip contributes to one datum in the analysis. The analyses are primarily between models across different conditions, focusing on quantifying the effects of OVR and SNR on WER and cpWER. It will also investigate on the transferability of model performance trends from synthetic to real-world data.

The aims of the statistical analysis are:

1. to quantify the effect of overlap ratio (OVR) on transcription performance while fixing other variables.
2. to quantify the effect of signal-to-noise ratio (SNR) on transcription performance while fixing other variables.
3. to analyse and compare performance of different ASR models on specific conditions
4. to evaluate whether performance trends observed on synthetic data transfer to real-world recordings.

**Outcome Measurement metrics**

The primary metrics for evaluating ASR performance are Word Error Rate (WER), concatenated permutation WER (cpWER) and Diarization-Invariant cpWER. WER is defined as:

$$\text{WER} = \frac{S + D + I}{N}, \tag{10.1}$$

where $S$, $D$, and $I$ are the numbers of substitutions, deletions, and insertions respectively, and $N$ is total number of words in the reference transcript. This measure the percentage of incorrect words relative to the reference transcript.

The cpWER is measured using the following method [2]:

1. Group the transcriptions generated and reference transcriptions by speaker.
2. Concatenate every transcription parts for each speaker (for both generated and reference) into a single string.
3. Compute the WER between the reference and generated transcriptions for all possible permutation of speaker assignments.
4. Use the minimum WER as the cpWER.

DI-cpWER is computed as cpWER if all speaker labels are correct [7].

WER is used for clips with no overlap (OVR = 0%) while cpWER is used for all clips except ones with no overlap (OVR = 0%) since WER does not apply when there are multiple speakers. DI-cpWER is used as diarization-invariant metric for all clips with overlapping, this is to evaluate the performance of ASR models in isolation from speaker assignment errors.

### General Analysis Principles

All hypothesis tests will be two-sided with significance level

$$\alpha = 0.05.$$

For each analysis, the response variable will be either WER or cpWER or DI-cpWER. Analysis is done with repeated-measures ANOVA.

For each fitted model, the following will be reported:

- group means and standard deviations;
- medians and interquartile ranges;
- estimated marginal means;
- 95% confidence intervals;
- $p$-values for main effects and interactions; and
- adjusted pairwise comparisons where relevant.

Pairwise comparisons with different models under constant conditions will be corrected using the Holm method.

Separate analyses will be conducted for WER, cpWER and DI-cpWER. For WER, let $\mathcal{S}_c$ denote the set of clips in condition $c$ and let $\mathrm{WER}_{i,m}$ denote the WER for clip $i$ and ASR model $m$. The model is

$$\overline{\mathrm{WER}}_{c,m} = \frac{1}{|\mathcal{S}_c|} \sum_{s \in \mathcal{S}_c} \mathrm{WER}_{i,m}$$

similarly for cpWER and DI-cpWER.

### Analysis of SNR Effect under Fixed Conditions

To isolate the effect of SNR, the primary analysis will fix:

- OVR = 14%,
- maximum concurrent speakers = 2,
- no preprocessing pipeline

If either effect is significant, post-hoc pairwise comparisons between SNR levels will be performed with Holm-adjusted $p$-values.

### Analysis of OVR Effect under Fixed Conditions

To isolate the effect of overlap ratio, the primary analysis will fix:

- SNR = 7.4 dB,
- maximum concurrent speakers = 2,
- a fixed preprocessing pipeline, and
- a fixed noise type.

The overlap levels considered will be 0%, 14%, 20%, and 40%. Let $y_{ikm}$ denote the metric for clip $i$, OVR level $k$, and ASR model $m$. The model is

$$y_{ikm} = \gamma_0 + \gamma_1 \,\mathrm{OVR}_k + \gamma_2 \,\mathrm{Model}_m + \gamma_3 \,(\mathrm{OVR}_k \times \mathrm{Model}_m) + \eta_{ikm}. \tag{10.2}$$

Again, $\mathrm{OVR}_k$ is treated as a categorical fixed effect. The null hypotheses are:

$$H_{0,3} : \text{no main effect of OVR on the metric,} \tag{10.3}$$

$$H_{0,4} : \text{no interaction between OVR and ASR model.} \tag{10.4}$$

If significant effects are detected, Holm-adjusted post-hoc pairwise comparisons between overlap levels will be performed.

### Distribution Analysis Within Each Condition

In addition to mean performance, the distribution of evaluation scores within each model-condition combination will be analysed to assess robustness and variability. For each model under each condition, the following descriptive statistics will be reported:

- mean;
- standard deviation;
- median;
- interquartile range;
- minimum and maximum; and
- selected quantiles.

These distributions will be visualised using boxplots and violin plots. This analysis is intended to identify whether certain conditions lead not only to worse average performance, but also to greater instability or spread in transcription accuracy.

Where relevant, Brown–Forsythe tests will be used as a formal comparison of variance across conditions.

### Transfer to Real-World Data

Transfer from synthetic to real-world data will be evaluated at selected operating points. The primary operating point will be

$$(\mathrm{OVR}, \mathrm{SNR}) = (14\%, 7.4 \text{ dB}),$$

and a secondary operating point will be

$$(\mathrm{OVR}, \mathrm{SNR}) = (20\%, 0 \text{ dB}).$$

For each operating point, ASR models will be evaluated on:

1. synthetic clips generated under the matched condition; and
2. real-world clips whose measured characteristics are closest to that condition.

For each operating point, transfer will be analysed using the model

$$y_{jdm} = \delta_0 + \delta_1 \,\mathrm{Domain}_d + \delta_2 \,\mathrm{Model}_m + \delta_3 \,(\mathrm{Domain}_d \times \mathrm{Model}_m) + \epsilon_{jdm}, \tag{10.5}$$

where $\mathrm{Domain}_d \in \{\text{synthetic}, \text{real}\}$.

The main quantities of interest are:

- the main effect of domain;
- the domain-model interaction; and
- the consistency of model rankings between synthetic and real-world data.

Ranking consistency will be summarised using Spearman rank correlation between model-wise mean scores on synthetic and real-world data.

### Optional Mixed-Effects Extension

If multiple clips share the same speaker, source recording, or meeting session, a linear mixed-effects model will be used instead of a simple linear model by including a random intercept for the relevant grouping factor. For example, if clips are clustered within recording $r$,

$$y_{ijmr} = \beta_0 + \beta_1 \,\mathrm{Condition}_j + \beta_2 \,\mathrm{Model}_m + \beta_3 \,(\mathrm{Condition}_j \times \mathrm{Model}_m) + u_r + \varepsilon_{ijmr}, \tag{10.6}$$

where

$$u_r \sim \mathcal{N}(0, \sigma_u^2).$$

### Assumption Checks

Residual plots and normal Q–Q plots will be inspected for each fitted model. Homogeneity of variance will also be assessed. If model assumptions are seriously violated, a non-parametric sensitivity analysis will additionally be reported.

### Reporting

Results will be reported separately for WER and cpWER. For each experiment, the following will be presented:

- condition-wise means, standard deviations, medians, and interquartile ranges;
- estimated marginal means from the fitted model;
- 95% confidence intervals;
- omnibus test results for main effects and interactions; and
- adjusted pairwise comparisons for predefined contrasts.

The primary conclusions will be based on:

1. whether decreasing SNR significantly increases WER and cpWER at fixed overlap;
2. whether increasing OVR significantly increases WER and cpWER at fixed SNR; and
3. whether the relative behaviour of ASR models at matched synthetic conditions is preserved on real-world data.

# Bibliography


[1] AMI. Ami meeting corpus. https://www.openslr.org/16/, 2007. Accessed: 2026-02-14.

[2] CHiME. Chime-6 challenge. https://www.openslr.org/150/, 2020. Accessed: 2026-02-14.

[3] Zhuo Chen, Takuya Yoshioka, Liang Lu, Tianyan Zhou, Zhong Meng, Yi Luo, Jian Wu, Xiong Xiao, and Jinyu Li. Continuous speech separation: dataset and analysis, 2020. URL https://arxiv.org/abs/2001.11482.

[4] Lauren Hadley, William Whitmer, W. Brimijoin, and Graham Naylor. Conversation in small groups: Speaking and listening strategies depend on the complexities of the environment and group. *Psychonomic Bulletin & Review*, 28, 10 2020. doi: 10.3758/s13423-020-01821-9.

[5] Zachary Ellis, Jared Joselowitz, Yash Deo, Yajie He, Anna Kalygina, Aisling Higham, Mana Rahimzadeh, Yan Jia, Ibrahim Habli, and Ernest Lim. Wer is unaware: Assessing how asr errors distort clinical understanding in patient facing dialogue, 2026. URL https://arxiv.org/abs/2511.16544.

[6] John Hughes. The problem with word error rate (wer). Speechmatics blog, 2025. URL https://www.speechmatics.com/company/articles-and-news/the-problem-with-word-error-rate-wer. Updated: 2025-09-22, accessed: 2026-04-17.

[7] Thilo von Neumann, Christoph Boeddeker, Marc Delcroix, and Reinhold Haeb-Umbach. Word error rate definitions and algorithms for long-form multi-talker speech recognition. *arXiv:2508.02112*, 2025. URL https://arxiv.org/pdf/2508.02112.

[8] Jean Carletta, Simone Ashby, Sebastien Bourban, Mike Flynn, Mael Guillemot, Thomas Hain, Jaroslav Kadlec, Vasilis Karaiskos, Wessel Kraaij, Melissa Kronenthal, Guillaume Lathoud, Mike Lincoln, Agnes Lisowska Masson, Iain McCowan, Wilfried Post, Dennis Reidsma, and Pierre Wellner. The ami meeting corpus: A pre-announcement. In *Lecture Notes in Computer Science*, 07 2005. ISBN 978-3-540-32549-9. doi: 10.1007/11677482_3.

[9] Shinji Watanabe, Michael Mandel, Jon Barker, Emmanuel Vincent, Ashish Arora, Xuankai Chang, Sanjeev Khudanpur, Vimal Manohar, Daniel Povey, Desh Raj, David Snyder, Aswin Shanmugam Subramanian, Jan Trmal, Bar Ben Yair, Christoph Boeddeker, Zhaoheng Ni, Yusuke Fujita, Shota Horiguchi, Naoyuki Kanda, Takuya Yoshioka, and Neville Ryant. Chime-6 challenge:tackling multispeaker speech recognition for unsegmented recordings, 2020. URL https://arxiv.org/abs/2004.09249.

[10] Gordon Wichern, Joe Antognini, Michael Flynn, Licheng Richard Zhu, Emmett McQuinn, Dwight Crow, Ethan Manilow, and Jonathan Le Roux. Wham!: Extending speech separation to noisy environments, 2019. URL https://arxiv.org/abs/1907.01160.

[11] Matthew Maciejewski, Gordon Wichern, Emmett McQuinn, and Jonathan Le Roux. Whamr!: Noisy and reverberant single-channel speech separation, 2020. URL https://arxiv.org/abs/1910.10279.

[12] Joris Cosentino, Manuel Pariente, Samuele Cornell, Antoine Deleforge, and Emmanuel Vincent. Librimix: An open-source dataset for generalizable speech separation, 2020. URL https://arxiv.org/abs/2005.11262.

[13] Özgür Çetin and Elizabeth Shriberg. Analysis of overlaps in meetings by dialog factors, hot spots, speakers, and collection site: insights for automatic speech recognition. In *Interspeech 2006*, pages paper 1915–Mon2A2O.6, 2006. doi: 10.21437/Interspeech.2006-91.

[14] Muqiao Yang, Naoyuki Kanda, Xiaofei Wang, Jian Wu, Sunit Sivasankaran, Zhuo Chen, Jinyu Li, and Takuya Yoshioka. Simulating realistic speech overlaps improves multi-talker asr, 2022. URL https://arxiv.org/abs/2210.15715.

[15] Olivier Galibert. Methodologies for the evaluation of speaker diarization and automatic speech recognition in the presence of overlapping speech, 08 2013.

[16] Thilo von Neumann, Christoph Boeddeker, Keisuke Kinoshita, Marc Delcroix, and Reinhold Haeb-Umbach. On word error rate definitions and their efficient computation for multi-speaker speech recognition systems. In *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5. IEEE, June 2023. doi: 10.1109/icassp49357.2023.10094784. URL http://dx.doi.org/10.1109/ICASSP49357.2023.10094784.

[17] Thilo von Neumann, Christoph Boeddeker, Marc Delcroix, and Reinhold Haeb-Umbach. Meeteval: A toolkit for computation of word error rates for meeting transcription systems, 2024. URL https://arxiv.org/abs/2307.11394.

[18] Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations, 2020. URL https://arxiv.org/abs/2006.11477.

[19] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision, 2022. URL https://arxiv.org/abs/2212.04356.

[20] Monica Sekoyan, Nithin Rao Koluguri, Nune Tadevosyan, Piotr Zelasko, Travis Bartley, Nikolay Karpov, Jagadeesh Balam, and Boris Ginsburg. Canary-1b-v2 & parakeet-tdt-0.6b-v3: Efficient and high-performance models for multilingual asr and ast, 2025. URL https://arxiv.org/abs/2509.14128.

[21] Desh Raj, Pavel Denisov, Zhuo Chen, Hakan Erdogan, Zili Huang, Maokui He, Shinji Watanabe, Jun Du, Takuya Yoshioka, Yi Luo, Naoyuki Kanda, Jinyu Li, Scott Wisdom, and John R. Hershey. Integration of speech separation, diarization, and recognition for multi-speaker meetings: System description, comparison, and analysis, 2020. URL https://arxiv.org/abs/2011.02014.

[22] Thilo von Neumann, Christoph Boeddeker, Tobias Cord-Landwehr, Marc Delcroix, and Reinhold Haeb-Umbach. Meeting recognition with continuous speech separation and transcription-supported diarization, 2024. URL https://arxiv.org/abs/2309.16482.

[23] Thilo von Neumann, Christoph Boeddeker, Marc Delcroix, and Reinhold Haeb-Umbach. Word error rate definitions and algorithms for long-form multi-talker speech recognition. *IEEE Transactions on Audio, Speech and Language Processing*, 33:3174–3188, 2025. ISSN 2998-4173. doi: 10.1109/taslpro.2025.3589862. URL http://dx.doi.org/10.1109/TASLPRO.2025.3589862.

[24] Samuele Cornell, Taejin Park, Steve Huang, Christoph Boeddeker, Xuankai Chang, Matthew Maciejewski, Matthew Wiesner, Paola Garcia, and Shinji Watanabe. The chime-8 dasr challenge for generalizable and array agnostic distant automatic speech recognition and diarization, 2024. URL https://arxiv.org/abs/2407.16447.

[25] Sanyuan Chen, Yu Wu, Zhuo Chen, Jian Wu, Jinyu Li, Takuya Yoshioka, Chengyi Wang, Shujie Liu, and Ming Zhou. Continuous speech separation with conformer, 2020. URL https://arxiv.org/abs/2008.05773.

[26] Naoyuki Kanda, Guoli Ye, Yashesh Gaur, Xiaofei Wang, Zhong Meng, Zhuo Chen, and Takuya Yoshioka. End-to-end speaker-attributed asr with transformer, 2021. URL https://arxiv.org/abs/2104.02128.

[27] Xuankai Chang, Naoyuki Kanda, Yashesh Gaur, Xiaofei Wang, Zhong Meng, and Takuya Yoshioka. Hypothesis stitcher for end-to-end speaker-attributed asr on long-form multi-talker recordings, 2021. URL https://arxiv.org/abs/2101.01853.

[28] Naoyuki Kanda, Xiong Xiao, Yashesh Gaur, Xiaofei Wang, Zhong Meng, Zhuo Chen, and Takuya Yoshioka. Transcribe-to-diarize: Neural speaker diarization for unlimited number of speakers using end-to-end speaker-attributed asr, 2022. URL https://arxiv.org/abs/2110.03151.

[29] Can Cui, Imran Ahamad Sheikh, Mostafa Sadeghi, and Emmanuel Vincent. Improving speaker assignment in speaker-attributed asr for real meeting applications, 2024. URL https://arxiv.org/abs/2403.06570.

[30] Desh Raj, Matthew Wiesner, Matthew Maciejewski, Leibny Paola Garcia-Perera, Daniel Povey, and Sanjeev Khudanpur. On speaker attribution with surt, 2024. URL https://arxiv.org/abs/2401.15676.

[31] Xianrui Zheng, Chao Zhang, and Philip C. Woodland. Dncasr: End-to-end training for speaker-attributed asr, 2025. URL https://arxiv.org/abs/2506.01916.

[32] Vassil Panayotov and Daniel Povey. Librispeech asr corpus. http://www.openslr.org/12, 2015. Accessed: 2026-02-14.

[33] VoxCeleb. Cstr vctk corpus. https://datashare.ed.ac.uk/handle/10283/3443, 2019. Accessed: 2026-02-14.

[34] Joachim Thiemann (IRISA-CNRS), Nobutaka Ito (University of Tokyo), and Emmanuel Vincent (Inria Rennes - Bretagne Atlantique). Demand: a collection of multi-channel recordings of acoustic noise in diverse environments. https://zenodo.org/records/1227121, 2013. Accessed: 2026-02-14.

[35] OpenAI. openai/faster-whisper. https://github.com/SYSTRAN/faster-whisper, 2024. Accessed: 2026-02-14.

[36] Meta AI. facebook/wav2vec2-large-960h. https://huggingface.co/facebook/wav2vec2-large-960h, 2020. Accessed: 2026-02-14.

[37] NVIDIA. nvidia/parakeet-tdt-0.6b-v3. https://huggingface.co/nvidia/parakeet-tdt-0.6b-v3, 2025. Accessed: 2026-02-14.

[38] NVIDIA. nvidia/canary-qwen-2.5b. https://huggingface.co/nvidia/canary-qwen-2.5b, 2025. Accessed: 2026-02-14.

[39] IBM Research. ibm-granite/granite-speech-3.3-8b. https://huggingface.co/granite-speech-3.3-8b, 2025. Accessed: 2026-02-14.

[40] Microsoft. microsoft/phi-4-multimodal-instruct. https://huggingface.co/microsoft/Phi-4-multimodal-instruct, 2025. Accessed: 2026-02-14.

[41] ESPnet. Espnet. https://github.com/espnet/espnet?tab=readme-ov-file#se-speech-enhancement-and-separation, 2024. Accessed: 2026-02-14.

[42] Ui-Hyeop Shin, Sangyoun Lee, Taehan Kim, and Hyung-Min Park. Separate and reconstruct: Asymmetric encoder-decoder for speech separation. *arXiv:2406.05983*, 2024. URL https://arxiv.org/pdf/2406.05983.

[43] Quan Wang, Hannah Muckenhirn, Kevin Wilson, Prashant Sridhar, Zelin Wu, John Hershey, Rif A. Saurous, Ron J. Weiss, Ye Jia, and Ignacio Lopez Moreno. Voicefilter: Targeted voice separation by speaker-conditioned spectrogram masking. *arXiv:1810.04826*, 2019. URL https://arxiv.org/pdf/1810.04826.

[44] Elizabeth Shriberg, Andreas Stolcke, and Don Baron. Observations on overlap: findings and implications for automatic processing of multi-party conversation. In *Interspeech*, 2001. URL https://api.semanticscholar.org/CorpusID:11947676.

[45] Joris Cosentino, Manuel Pariente, Samuele Cornell, Antoine Deleforge, and Emmanuel Vincent. Handling trade-offs in speech separation with sparsely-gated mixture of experts, 2023. URL https://arxiv.org/abs/2211.06493.

[46] Wu YH, Stangl E, Chipara O, Hasan SS, Welhaven A, and Oleson J. Characteristics of real-world signal to noise ratios and speech listening situations of older adults with mild to moderate hearing loss. *Ear Hear*, 39(2):293–304, March 2018. doi: 10.1097/AUD.0000000000000486.